

Koku.

Parfum Kesif & Sosyal Platform

Teknik Mimari Dokumantasyonu

ASP.NET Core 9 | PostgreSQL | EF Core | Docker

1. Backend Mimarisi
2. Frontend / UI
3. Veritabani
4. Deployment & CI/CD

01



Backend Mimarisi

ASP.NET Core 9 MVC - Controllers, Models, Services

1.1 Genel Mimari

Proje, ASP.NET Core 9 MVC mimarisini kullanmaktadır. İstek akisi şu katmanlardan geçer:

Kullanıcı / Tarayıcı	HTTP istegi gönderir
v	
View (.cshtml)	Razor syntax ile HTML üretir, kullanıcıya gönderir
v	
Controller	İstegi karşılar, iş mantığı işletir, View'a veri gönderir
v	
Service / Helper	Karmaşık iş mantığı (onere motoru, bildirim, arama)
v	
Data (EF Core)	AppDbContext üzerinden PostgreSQL ile konuşur

Program.cs - Tüm servislerin kayıt edildiği ve middleware pipeline'inin kurulduğu giriş noktasıdır.

Teknoloji Stack

Teknoloji	Açıklama	Kategori
ASP.NET Core 9	Web framework - MVC mimarisi	Backend
C# 13	Primary constructor, collection expressions	Backend
EF Core 9	ORM - Code-First, Migrations	ORM
Npgsql	PostgreSQL EF Core provider	ORM
BCrypt.Net	Sıfır hash'leme (bcrypt algoritması)	Güvenlik
Cookie Auth	7 günlük sliding expiration session	Güvenlik
pg_trgm	Fuzzy arama için PostgreSQL extension	Arama

1.2 Controllers

Her controller bir ozellik grubundan sorumludur. [Authorize] attribute ile korunanlar login gerektirmektedir.

Controller	Sorumluluk	Auth
AuthController	Login, Register, Logout - cookie yazar/siler	Acik
HomeController	Karsilama ve hata sayfasi	Acik
PerfumesController	Parfum listesi, detay, arama, yeni parfum ekleme	[Authorize]*
FeedController	Ana akis - takip edilenlerin aktiviteleri	[Authorize]
CollectionController	Owned/Wishlist/Tried toggle	[Authorize]
ReviewsController	Yorum olusturma ve silme	[Authorize]
LikesController	Begeni toggle - AJAX ile sayfasiz	[Authorize]
FollowsController	Takip et / birak	[Authorize]
CommunitiesController	Topluluk CRUD, uye yonetimi, gonderi	[Authorize]
MessagesController	Ozel mesajlasma, konusma gorunumu	[Authorize]
NotificationsController	Bildirim listesi, okundu isareti	[Authorize]
DailyLogController	Gunluk koku kaydi	[Authorize]
ProfileController	Profil sayfasi, tat profili, kullanici arama	Karma

* Sadece Create (parfum ekleme) action'lari [Authorize] gerektirir; listeleme ve detay herkese aciktir.

1.3 Models - Veritabanı Sınıfları

Her model sınıfı bir veritabanı tablosuna karşılık gelir. EF Core bu sınıfları okuyarak tabloları oluşturur.

Model	Açıklama
User	Kullanıcı - kullanıcı adı, email, BCrypt hash, bio, avatar
Perfume	24.000+ parfüm - isim, marka, notalar, accord'lar, puan
Collection	Kullanıcı-parfüm ilişkisi - Owned / Wishlist / Tried statüleri
Review	Yorum ve puan (1-5 yıldız), kullanıcı-parfüm unique
Like	Begeni - kullanıcı-parfüm unique index
Follow	Takip ilişkisi - iki farklı User foreign key
DailyLog	Kaç sprey, hangi gün, hangi parfüm kaydedildi
Community	Topluluk - isim, açıklama, kurucu (OwnerId)
CommunityMember	Üyelik - Role alanı: 'admin' veya 'member'
CommunityPost	Topluluk gönderisi, opsiyonel parfüm bağlantısı
Message	Özel mesaj - Sender, Recipient, IsRead
Notification	Bildirim - takip/begeni/yorum olaylarında oluşturulur

1.4 Services

ICurrentUserService

Cookie'deki kullanıcı ID'sini her controller'dan kolayca okumay sağlar. HttpContext.User.Claims'den NameIdentifier claim'ini parse eder. Dependency injection ile constructor'a inject edilir.

NotificationService

Takip, beğeni ve yorum gibi olaylar gerçekleştiğinde otomatik Notification kaydı oluşturur. Her controller aksiyon sonrasında bu servisi çağırır, böylece bildirim mantığı controller'lardan ayrılmış olur.

UserTasteProfileService - Hibrit Oneri Motoru

En karmaşık servis. İki algoritmayı birleştirir:

- İçerik Tabanlı (%70): Kullanıcının beğenileri, koleksiyonu, yorumları ve günlük log'larından ağırlıklı sinyal vektörü çıkarır. Aday parfümlerin accord vektörü ile cosine similarity hesaplar.
- İsbirlikçi (%30): Aynı parfümleri seven diğer kullanıcıları bulur (min 2 ortak etkileşim), onların beğenip benim etkileşmediğim parfümleri öneriye ekler.

Sinyal Ağırlıkları

Etkileşim Turu	Ağırlık
Owned (koleksiyona eklendi)	+2.5
Begeni	+2.0
Wishlist	+1.5
5 yıldız yorum	+3.0
4 yıldız yorum	+1.5
3 yıldız yorum	0.0 (notr)
2 yıldız yorum	-1.5
1 yıldız yorum	-3.0 (negatif sinyal)
Daily Log (her gün)	+0.3 (max 3.0)

Filters/ - UnreadCountFilter

Action filter olarak tüm korumalı sayfalara global eklenmiştir. Her sayfa yüklendiğinde okunmamış bildirim sayısını hesaplar ve ViewData'ya yazar. _Layout.cshtml bu değeri sidebar'daki badge'de gösterir.

Helpers/

Dosya	Açıklama
AccordVisual.cs	Her accord turüne renk + ikon atar (woody -> kahverengi, citrus -> sarı)
NoteVisual.cs	Her koku notasına renk + ikon atar
PerfumeSearchQuery.cs	pg_trgm fuzzy arama - ILIKE + word_similarity, token bazlı AND zinciri
SearchNormalizer.cs	Türkçe karakter normalizasyonu (ğ->g, ş->s), token üretimi

02



Frontend / UI

Razor Views, CSS, JavaScript

2.1 Genel Yapı

Frontend, sunucu tarafında render edilen Razor View'lara dayanır. Sayfa istegi geldiginde controller veriyi ViewModel'e paketler, View bu veriyi HTML'e donusturur ve tarayiciya gönderir. JavaScript sadece dinamik kisimler icin kullanilir (AJAX, toggle, autocomplete).

Mimari: Server-Side Rendering (SSR) + Partial AJAX. Tam sayfa yenilemesi sadece navigasyonda, filtreler ve toggle'lar AJAX ile ca

Shared/_Layout.cshtml - Ortak Iskelet

Tum sayfalar bu Layout'a sarilir. Icerir:

- Sol sidebar: logo, navigasyon linkleri, kullanıcı adı, çıkış butonu
- ViewData['ActivePage'] ile aktif menu ogesi vurgulanır
- Bootstrap 5, jQuery, Bootstrap Icons CDN yuklemeleri
- @RenderBody() - her sayfanin kendi icerigi buraya gelir
- @RenderSection('Scripts') - sayfaya ozel JS en sonda yuklenir
- Okunmamis bildirim sayaci (UnreadCountFilter'dan gelen ViewData)

_ViewImports.cshtml ve _ViewStart.cshtml

_ViewImports: Tum view'lara otomatik @using ve Tag Helper import'larini ekler (asp-controller, asp-action vb. Tag Helper'lar buradan aktif olur).

_ViewStart: Her view icin varsayılan layout'u belirler (_Layout.cshtml).

2.2 View Dosyalari

Her controller klasorune karsilik gelen View klasoru vardir. Sayfalar Razor syntax ile C# + HTML birlikte yazar.

View	Sayfa Adi	Satir
Perfumes/Index.cshtml	Kesfet - filtreler, AJAX arama, chip'ler	431
Profile/View.cshtml	Profil - tat profili, istatistikler	419
Feed/Index.cshtml	Ana Akis - aktiviteler + oneriler	378
DailyLog/Index.cshtml	Gunluk Log - takvim + autocomplete	334
Perfumes/Details.cshtml	Parfum Detay - notalar, yorumlar	249
Communities/Details.cshtml	Topluluk Detay - gonderiler, uyeler	164
Notifications/Index.cshtml	Bildirimler	136
Shared/_Layout.cshtml	Ortak iskelet - sidebar, nav	134
Collection/Index.cshtml	Koleksiyonum - 3 sekme	122
Perfumes/_PerfumeGrid.cshtml	Grid Partial - sadece AJAX'ta doner	104
Perfumes/Create.cshtml	Yeni Parfum (admin)	100
Profile/Search.cshtml	Kullanici Arama	84
Communities/Index.cshtml	Topluluk Listesi	76
Messages/Conversation.cshtml	Konusma gorunumu	71
Likes/Index.cshtml	Begendiklerim	58
Auth/Login.cshtml	Giris formu	~40
Auth/Register.cshtml	Kayit formu	~40

2.3 CSS - site.css (1172 satir)

Tum ozel stiller tek bir dosyada toplanmistir. Bootstrap 5'in uzerine yazilmistir.

Renk Degiskenleri	--amber, --dark-bg vb. CSS custom properties. Tum renkler buradan yonetilir.
Sidebar	Sol navigasyon, kullanci avatari, aktif link vurgusu, hover efektleri.
Parfum Kartlari	.parfum-card - kose yuvarlama, golge, hover animasyonu.
Filter Sistemi	.filter-chip, .filter-card, .brand-dropdown, .filter-active-bar, .filter-collapsible.
Accord Chip'leri	.accord-chip - tiklanabilir renkli etiketler. Her accord'un rengi AccordVisual.cs'den...
Nota Chip'leri	.note-chip - koku notalarina gore renkli ikonlu etiketler.
Feed Kartlari	Aktivite satirlari, paylasim kutusu, modal stilleri.
Mesaj Baloncuklari	Sent (sag, amber) / received (sol, gri) ayrimi.
Profil Sayfasi	Avatar, istatistik kutulari, tat profili radar grafigi.
Responsive	867. satirdan itibaren mobil breakpoint'ler (@media max-width).
Nota & Akor Chip'leri	1076. satirdan - detay sayfasindaki renk/ikon sistemi.

2.4 JavaScript Dosyaları

wwwroot/js/site.js - 44 satır

Tüm sayfalarda çalışan genel JS. Begeni butonu AJAX toggle'i burada tanımlıdır. Tiklanınca sayfa yenilenmeden kalp ikonu ve sayac güncellenir.

```
// Begeni toggle - AJAX, sayfa yenilemeden
document.querySelectorAll("[data-like-toggle]").forEach(btn => {
  btn.addEventListener("click", async () => {
    const res = await fetch("/Likes/Toggle", { method:"POST", ... });
    // İkon ve sayacı güncelle
  });
});
```

wwwroot/js/perfume-autocomplete.js - 171 satır

Günlük Log, Topluluk gönderisi ve Feed modal'ındaki parfüm arama kutusu. Yazarken /Perfumes/Search?q=... endpoint'ini çağırır, sonuçları dropdown olarak gösterir, seçilince hidden input'a ID'yi yazar.

```
// Autocomplete akisi
input.addEventListener("input", async () => {
  const res = await fetch(`/Perfumes/Search?q=${q}&limit=8`);
  const data = await res.json();
  renderDropdown(data); // isim + marka listesi göster
});
```

Sayfa Basi Inline JS (@section Scripts)

Her view'in kendi JS'i @section Scripts { } bloğu içinde yazılır ve _Layout'taki @RenderSection("Scripts") sayesinde sayfanın sonunda yüklenir. Örnek sayfalar ve JS'leri:

Kesfet (Index.cshtml)	AJAX filtre, accord chip multi-select, brand dropdown, aktif bar
Feed (Index.cshtml)	Paylaşım modal'i, parfüm autocomplete, begeni AJAX
DailyLog (Index.cshtml)	Takvim seçimi, parfüm autocomplete modal
Details (Details.cshtml)	Koleksiyon toggle, begeni AJAX

Dis Kutuphaneler (wwwroot/lib/)

Kutüphane	Kullanım
Bootstrap 5	Grid, butonlar, modal, dropdown, badge
Bootstrap Icons	bi-search, bi-heart, bi-droplet vb. ikonlar
jQuery	AJAX istekleri, DOM manipülasyonu
jQuery Validation	Client-side form validasyonu
jQuery Validation Unobtrusive	Data Annotation'larını client'a tasir ([Required] -> form hatası)

03

[DB]

Veritabani

PostgreSQL 16 + EF Core Code-First + Migrations

3.1 Veritabani Mimarisi

Proje PostgreSQL 16 kullanmaktadır. EF Core Code-First yaklasimi ile veri modeli C# siniflarindan turetilir. Veritabani dogrudan duzenlenmez; tum degisiklikler Migration dosyalari araciligiyla uygulanir.

Baglanti: Npgsql EF Core Provider üzerinden. Connection string .env dosyasindan okunur, asla koda yazilmaz.

AppDbContext.cs - Merkez Sinif

- 12 adet DbSet<T> tanimi - her biri bir tabloya karsilik gelir
- Unique constraint'ler: kullanıcı adi, email, begeni, yorum, takip iliskileri
- Cascade/Restrict delete kurallari - orpan kayıt olusmasin diye
- Follow modeli icin 2 ayri User iliskisi (circular reference onlemi)
- Notification modeli icin Recipient ve Actor seklinde 2 User FK
- Perfume icin GENERATED STORED kolon konfigurasyonu (SearchKey)

Migrations - 3 Asama

Migration	Tarih	Ne Degistirdi
InitialCreate	08.05.2026	Tum tablolar sifirdan olusturuldu
CollectionMultiStatus	25.05.2026	Bir parfum ayni anda Owned+Wis
AddPerfumeSearchIndex	25.05.2026	pg_trgm, SearchKey GENERATE

3.2 Tablo İlişkileri

Tablolar arası ilişkiler (FK - Foreign Key bağlantıları):

İlişki	Açıklama
User -> Collection	Bir kullanıcının birden fazla koleksiyon girişi olabilir
User -> Review	Bir kullanıcı bir perfume yalnızca bir yorum yapabilir (unique)
User -> Like	Bir kullanıcı bir parfümi yalnızca bir kez beğenir (unique)
User -> Follow	Takip için iki FK: Follower ve Following (her ikisi de User)
User -> DailyLog	Bir kullanıcının günlük koku kayıtları
User -> Notification	Recipient ve Actor olmak üzere iki FK
User -> Message	Sender ve Recipient olmak üzere iki FK
Community -> Member	CommunityMember üzerinden çoktan-coka - Role: admin/member
Community -> Post	Topluluk gönderileri, opsiyonel Perfume bağlantısı
Perfume -> Collection	Hangi parfüm hangi koleksiyonda (Status ile birlikte unique)
Perfume -> Review	Parfüm yorumları ve puanları
Perfume -> Like	Parfüm beğenileri

3.3 Arama Altyapısı - pg_trgm

Arama motoru PostgreSQL'in pg_trgm extension'i üzerine kurulmuştur. Yazı hatalarını tolere eden (fuzzy) ve Türkçe karakter destekli bir arama sağlar.

SearchKey - GENERATED STORED Kolon

Her parfüm kaydı için otomatik olarak hesaplanan normalize edilmiş metin kolonu. İçerir: İsim + Marka + Top/Middle/Base Notalar + 5 Accord. perfume_normalize() fonksiyonu ile Türkçe karakterler temizlenir, küçük harfe çevrilir.

SQL:

```
ALTER TABLE "Perfumes" ADD COLUMN "SearchKey" text
GENERATED ALWAYS AS (
    perfume_normalize(
        coalesce("Name", "") || " " || coalesce("Brand", "") || " " ||
        coalesce("TopNotes", "") || " " || coalesce("Accord1", "") || ...
    )
) STORED;
```

Arama Mantığı - PerfumeSearchQuery.cs

Kullanıcının yazdığı metin tokenlara bölünür. Her token için iki koldan biri sağlanmış olmalı: ILIKE (substring eşleşmesi) VEYA word_similarity > 0.4 (fuzzy eşleşmesi). Tokenlar AND ile zincirlenir.

C#:

```
foreach (var token in tokens) {
    query = query.Where(p =>
        EF.Functions.ILike(p.SearchKey, "%token%") ||
        EF.Functions.TrigramsWordSimilarity(token, p.SearchKey) > 0.4
    );
}
```

GIN Index

Büyük veri seti üzerinde hızlı trigram eşleşimi için GIN (Generalized Inverted Index) kullanılır. 24.000 parfüm üzerinde milisaniye düzeyinde arama sağlar.

DbInitializer.cs - Veri Yükleme

Uygulama ilk başladığında çalışır ve şu adımları izler:

- db.Database.EnsureCreated() - DB yoksa tablo semayı oluşturur
- Perfume tablosu boş ise CSV'yi okur, 24.000 parfümü 500'er batch ile ekler
- Users tablosu boş ise SeedMockData çalışır: 6 kullanıcı, yorumlar, beğeniler, takipler, 2 topluluk, mesajlar oluşturulur
- Zaten veri varsa hiçbir şey yapmaz (idempotent)

04



Deployment & CI/CD

Docker + GitHub Actions + Cloudflare + Linux VDS

4.1 Deployment Mimarisi

Kod production'a ulasmak icin su katmanlardan gecer:



4.2 GitHub Actions Workflow

Dosya: .github/workflows/deploy.yml

deploy.yml:

```
name: Deploy to VDS

on:
  push:
    branches: [production] # sadece production branch tetikler
  workflow_dispatch:      # elle de tetiklenebilir

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: SSH and deploy
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.VDS_HOST } # sunucu IP
          username: ${ secrets.VDS_USER } # SSH kullanicisi
          key: ${ secrets.VDS_SSH_KEY } # private key
          script: /home/parfum/web_parfum/deploy.sh
```

Secrets: VDS_HOST, VDS_USER ve VDS_SSH_KEY GitHub repository Settings > Secrets'ta saklanir. Kodda gozukmez.

Tetikleme Kosullari

- production branch'ine her push otomatik deploy basliir
- main, develop gibi diger branch'lere push deploy etmez
- GitHub arayuzunden workflow_dispatch ile elle de tetiklenebilir

4.3 Docker Yapısı

Uygulama iki Docker container'dan oluşur:

docker-compose.yml - Lokal Gelistirme

Lokal (docker-compose.yml):

```
services:
  app:
    build: ./web_parfum_dotnet
    ports: ["0.0.0.0:8080:8080"]
    depends_on: [db]
  db:
    image: postgres:17-alpine
    ports: ["127.0.0.1:5433:5432"]
    volumes: [postgres_data:/var/lib/postgresql/data]
```

docker-compose.prod.yml - Production

Production (docker-compose.prod.yml):

```
services:
  app:
    build: ./web_parfum_dotnet
    restart: unless-stopped # sunucu yeniden baslarsa oto kalkar
    ports:
      - "127.0.0.1:8080:8080" # sadece localhost'a bind
    networks: [shared-db] # mevcut postgres container'i paylas

networks:
  shared-db:
    external: true # onceden var olmalı
```

Production'da Nginx/Cloudflare reverse proxy üzerinden geldiği için port doğrudan dışarıya açılmaz (127.0.0.1 bind). Bu, doğrudan IP ile erişimleri engeller.

4.4 Cloudflare & SSL

Domain Cloudflare üzerinden yönetilmektedir.

DNS Yapisi

- A kaydi: domain -> sunucu IP'si, bulut turuncu (proxied)
- Cloudflare ziyaretci ile HTTPS, sunucu ile HTTP konusur (Flexible mode)
- Sunucu firewall'da yalnızca port 80 acik - Cloudflare'den gelen trafik

ForwardedHeaders Middleware

Cloudflare proxy arkasında çalışıldığı için uygulamanın gerçek IP'yi ve sema'yi (https) doğru okuyabilmesi için ForwardedHeaders middleware kullanılır. Bu olmadan cookie'lerin Secure flag'i yanlış kurulur ve giriş yapılamaz.

```
// Program.cs
builder.Services.Configure<ForwardedHeadersOptions>(options => {
    options.ForwardedHeaders = ForwardedHeaders.XForwardedFor
        | ForwardedHeaders.XForwardedProto;
    options.KnownNetworks.Clear(); // Cloudflare IP'lerine güven
    options.KnownProxies.Clear();
});
// ONEMLI: UseForwardedHeaders() UseAuthentication'dan ONCE olmalı
app.UseForwardedHeaders();
```

HttpsRedirection Production'da Kapalı

Cloudflare'in Flexible SSL modunda, sunucu zaten HTTP aldığı için UseHttpsRedirection() redirect loop oluşturur. Bu nedenle production ortamında devre dışı bırakılmıştır:

```
if (!app.Environment.IsProduction()) {
    app.UseHttpsRedirection(); // Sadece lokal geliştirmede aktif
}
```

4.5 Ortam Degiskenleri (.env)

Hassas bilgiler .env dosyasinda tutulur, asla Git'e commit edilmez (.gitignore'da listelenmistir).

.env.example:

```
# .env.example - gercek degerler .env dosyasinda
POSTGRES_PASSWORD=guclu_bir_sifre
POSTGRES_DB=web_parfum
POSTGRES_USER=postgres
APP_PORT=80
DB_HOST=postgres
ASPNETCORE_ENVIRONMENT=Production
```

Degisken	Aciklama
POSTGRES_PASSWORD	PostgreSQL sifre - zorunlu, bos birakilamaz
POSTGRES_DB	Veritabani adi (varsayilan: web_parfum)
POSTGRES_USER	DB kullanici adi (varsayilan: postgres)
APP_PORT	Ugulamanin dinledigi port (production'da 80)
DB_HOST	PostgreSQL container adi (Docker network'te cozumlenir)
ASPNETCORE_ENVIRONMENT	Production / Development - middleware davranisini belirler

Guncelleme Sureci

Production'da yeni kod deploy etmek icin tek yapilmasi gereken:

```
git add .
git commit -m 'Aciklayici commit mesaji'
git push origin production

# GitHub Actions otomatik devreye girer:
# 1. SSH baglantisi kurar
# 2. deploy.sh calistirir
# 3. git pull && docker compose up -d --build
# 4. Yeni container ayaga kalkar, site guncellenir
```